

Supervised Deep Learning for Cross-Lingual Question Answering in Low-Resource Language

M. Abdullah Bin Asim
Department of DataScience/Artificial
Intelligence
FAST-NUCES
Islamabad, Pakistan
i232621@isb.nu.edu.pk

Maimoona Hannan
Department of DataScience/Artificial
Intelligence
FAST-NUCES
Islamabad, Pakistan
i232544@isb.nu.edu.pk

Fatima Tu Zahra
Department of DataScience/Artificial
Intelligence
FAST-NUCES
Islamabad, Pakistan
i232616@isb.nu.edu.pk

Abstract—This research examines extractive question answering(QA) for a low resource Language, Urdu by employing the multilingual Transformer models on the UQA dataset obtained from SQuAD 2.0 through EATS translation method. We replicate baseline outcomes for mBERT and XLM-RoBERTa, and then further replication with specific experiments: fixing mBERT training configuration issues, applying parameter-efficient LoRA finetuning for XLM-R, conducting zero-shot transfer to XQuAD Hindi and carrying out sequential crosslingual transfer. There was no major difference in XLM-R baseline and reproduced results however mBERT performance was enhanced by 8.61% and 24.86% increase in F1 score and EM(Exact Measure), underscoring the significance of training setup. In our scenario, LoRA shows inferior performance by 23.12% and 33.78% drop in F1 score and EM(Exact Match) compared to full fine-tuning whereas the model trained on URDU exhibits significant cross-linguistic generalization to Hindi.

Keywords—Urdu NLP; Extractive Question Answering; Transformers; Cross-lingual Transfer; LoRA; EM

I. INTRODUCTION

When it comes to Natural Language Processing, one of the most important tasks is Question Answering (QA). This is where a system reads a passage and gives an answer by finding the right part of the text. Recently, models like multilingual Transformers have made big progress in QA for many languages. However, how well they work and how reliable they are depend a lot on having large, well-organized datasets that people have carefully labeled. Urdu is a language spoken by over 70 million people, but it doesn't have as many resources for QA as other languages. This project is about repeating the results of a baseline study for the UQA dataset, which is the Urdu version of SQuAD 2.0. Initially, we wanted to see how well a model trained on English data would work on Urdu data, but for this project, we focused on getting the same results as the original study using publicly available tools and training materials. Because we had limited time and computer power, we didn't train a new model and instead used the results from a model called XLM-RoBERTa. Our report includes a step-by-step guide on how the dataset was made and how the model was trained, a clear explanation of the settings and parameters we used, a comparison of our results with the original study, and a discussion on how reliable our results are and why they might be different from the original study. To

measure how well the model works, we use two common metrics: Exact Match (EM) and token-level F1. EM is simple - it's 1 if the model's answer is exactly the same as the correct answer, and 0 if it's not. Token-level F1 is a bit more complicated. It looks at how many of the words in the model's answer are also in the correct answer, and how many of the words in the correct answer are in the model's answer. Then it calculates a score based on these two numbers. In datasets where some questions don't have answers, like SQuAD 2.0, the model needs to be able to say "no answer" for those questions. We used the same metrics as the SQuAD-style evaluation to compare our results. Note that we didn't change proper nouns, names, positions, places, emails, phone numbers, dates, times, and short phrases under 4-5 words, as per the instructions. We also retained formatting-related symbols and aimed for a similar writing style, vocabulary, sentence structure, and punctuation as the provided human reference paragraphs. In our reproduction notebook, we filtered out unanswerable examples prior to training and evaluation; therefore, the reported EM/F1 should be interpreted as answerable-only performance.

Let's take a closer look at what happened when we trained mBERT and XLM-R. It took around 18 hours to get the job done on Google Colab T4 hardware. But that was just the beginning. We wanted to push the boundaries of what these models could do, so we designed four experiments to test their limits. The first experiment was all about fixing the flaws in mBERT's configuration to see just how well it could perform on UQA. We were curious to know what the real potential of mBERT was. The second experiment was a bit different. We used something called LoRA, or Low-Rank Adaptation, to fine-tune XLM-RoBERTa. We wanted to know if this approach could match the results of full fine-tuning, but with much less computational power. The third experiment took things a step further. We tested the LoRA-trained model on XQuAD Hindi to see how well it could handle zero-shot cross-lingual transfer. And finally, the fourth experiment involved pre-training on English SQuAD and then fine-tuning on Urdu UQA. We were interested in finding out if starting with English QA could improve performance on Urdu tasks. Throughout all these experiments, we used the same evaluation method: Exact Match and token-level F1 on the UQA answerable-only

validation set, which had 5,811 unique questions. By doing so, we could compare the results and see what worked best.

1.1 Research Questions

RQ1: What is the impact of prior fine-tuning on English QA datasets on the performance of models subsequently fine-tuned on Urdu QA datasets?

RQ2: How do different multilingual transformer models (mBERT vs XLM-RoBERTa) compare in their effectiveness for cross-lingual transfer learning in Urdu extractive QA?

RQ3: Does cross-lingual transfer learning reduce the amount of Urdu training data required to achieve competitive extractive QA performance?

II. BACKGROUND AND RELATED WORK

The foundational paper “Arif et al. (2024)” presents UQA, an extensive Urdu Question Answering dataset sourced from SQuAD 2.0. The main difficulty in translating SQuAD-style QA lies in maintaining answer spans: the translated answer should exist as a continuous substring within the translated context for extractive QA to remain clearly defined.

2.1 Dataset and Translation Technique (EATS)

The dataset adheres to the SQuAD 2.0 format, featuring questions that are either answerable or unanswerable. The UQA repository outlines the application of the EATS method (Enclose to Anchor, Translate, Seek): Enclose to Anchor: highlight the answer span within the English context using a delimiter (in the repository notebooks, the marker is ‘••’). Translate: convert title/question/context while striving to maintain the anchors during translation (the repository utilizes the SeamlessM4T model). Find: identify the anchors within the translated context, eliminate them, and calculate the Urdu answer text along with its character-level starting position.

2.2 Size of the Dataset and Its Divisions

The released UQA dataset is provided in a SQuAD-like format containing training and validation partitions. Our initial proposal outlined the dataset broadly (approximately 141k total QA pairs).

2.3 Models and Documented Baselines

In this report, we utilize the model benchmark table from the README and our replicated metrics from the training notebook for model comparison.

TABLE I. . BASELINE UQA RESULTS

Dataset	Exact Match	F1 Score
mBert	45.50	64.72
XLM-RoBERTa (base)	65.67	78.00
mT5 -Small	52.37	67.72
mT5 -Large	71.26	84.20
XLM-RoBERTa-Large	72.24	84.42
XLM-RoBERTa-XL	74.56	85.99

In this report, we compare our reproduced results to the matching base model entry from the README.

2.4 Practical Limitations in Our Replication

The initial paper and repository evaluate various model sizes, featuring larger versions that demand significant GPU memory and training duration. For our reproduction, we utilized a Kaggle GPU configuration (T4 $\times$ 2, 15 GiB each) and concentrated on one robust baseline: XLM-RoBERTa base. Even though the repository contains an mT5 notebook, we did not train mT5 in our setup because of computational limitations.

2.5 Principal Contributions of the Baseline Paper

Based on the README from the repository and the abstract of the paper, the key contributions of Arif et al. (2024) can be condensed as follows:

A comprehensive Urdu QA dataset (UQA) created from SQuAD 2.0, facilitating uniform training and assessment for extractive QA in Urdu.

A pragmatic translation method (EATS) that is tailored to maintain answer spans in the translated context, crucial for extractive QA.

3) Standards for multilingual models (mBERT, XLM-R variants, mT5 variants) that set benchmarks for upcoming research and emphasize the advantages of model size.

The README further mentions that the creators assessed various translation options (Google Translator vs. SeamlessM4T) during the dataset creation, highlighting that translation quality is considered a primary design element instead of a secondary consideration.

III. PROPOSED MODEL AND METHODS

To leverage cross-lingual transfer learning, we recommend employing multilingual transformer-based deep neural networks. The models will be initialized using pretrained multilingual weights (mBERT or XLM-R). English QA datasets can serve for warm-starting, while the Urdu QA dataset will be utilized for fine-tuning. To comprehend complex contextual patterns, the model will accept a question and a context paragraph as input, process them through multiple transformer layers, and generate probability distributions for the starting and ending positions of the answer span within the context. Multilingual tokenizers that manage Urdu script, like the mBERT tokenizer or XLM-R tokenizer, will tokenize questions and contexts for input representations. Each token is represented by a numerical ID derived from the model's vocabulary. Subsequently, an embedding layer converts these IDs into compact vector embeddings. A single input sequence is formed by combining the query and context with special tokens ([CLS], [SEP]). Token positions for the start and end within the tokenized context are utilized to indicate answer spans in the training dataset.

3.2 Summary of Network Structure

The models consist of multiple transformer layers featuring self-attention mechanisms (12 layers for the mBERT base and 12 layers for the XLM-R base). Every

layer employs feed-forward networks featuring non-linear activations (GELU activation function) following the application of multi-head self-attention. The entire network utilizes residual connections and layer normalization. The final tier includes two categorization heads that forecast start and end positions employing softmax activation across the context tokens.

3.3 Strategy for Cross-Lingual Transfer

The following strategies will be assessed: Start models using multilingual pretrained weights (XLM-R or mBERT). You can choose to warm up by honing your English QA datasets (SQuAD) Adjust the Urdu QA dataset (UQA). Compare cross-lingual transfer fine-tuning with direct fine-tuning. Cross-entropy loss for start and end position tokens is the training method's loss function. For both start and end positions, the model generates probability distributions over all tokens; the loss is calculated as the total of the cross-entropy losses for each position. AdamW optimizer with learning rate scheduling (linear warmup and linear decay) is used for optimization. To avoid overfitting, training will end if the validation F1 score does not improve for a predetermined period of epochs. This method is appropriate for a number of reasons related to Goodfellow's deep learning principles:

- 1) Representation Learning: Without the need for manually created features, deep neural networks are able to automatically learn hierarchical language representations from input. From token-level characteristics to semantic connections between inquiry and context, the transformer layers acquire progressively more abstract representations of the input text.

- 2) Managing High-Dimensional Data: Large vocabularies and intricate syntactic and semantic patterns make language data intrinsically high-dimensional. By developing distributed representations that more effectively capture statistical regularities in the data than conventional techniques, deep learning overcomes the curse of dimensionality.

- 4) Transfer Learning: Deep models that have been pretrained on extensive multilingual datasets have acquired universal linguistic characteristics that are applicable to all languages. This solves the issue of data scarcity by enabling the model to transfer information from high-resource languages (English) to low-resource languages (Urdu).

- 5) Depth and Expressiveness: Transformer models' depth (many layers) enables them to combine simple functions into complex functions that are capable of capturing intricate semantic linkages and long-range dependencies between queries and contexts. This is crucial since comprehension of QA necessitates combining data from several sections of the text.

3.4 Training Pipeline

The complete pipeline was not constructed from scratch. Instead, we executed and adjusted the notebooks and scripts provided by the author, available in the public 'uqa-project' directory. Hugging Face libraries are utilized in the process for training, evaluation, tokenization, and data loading.

- 1) Anchoring and Data Cleaning (Clean.ipynb): The initial English SQuAD 2.0 JSON files are imported into the cleaning notebook, which prepares them for translation and recovery of spans based on anchors. Significant actions recorded in the repository include; Long-context handling entails exporting contexts exceeding 1000 characters to text files for manual paragraph division (in two phases, depending on the presence of line breaks). Various Unicode punctuation marks are standardized (e.g., en-dash/em-dash → '--', smart quotes eliminated, semicolon converted to Urdu '؛'). For every QA instance, the answer text segment is highlighted within the context using the marker '••' to facilitate its identification post-translation. In SQuAD 2.0's unanswerable questions, the cleaning code utilizes 'plausible_answers' as an anchor (when available), allowing the translated context to maintain a stable anchor position even if the question is marked unanswerable. This method is significant because the direct translation of (context, answer) pairs frequently disrupts extractive QA since the translated answer might not be present word-for-word within the translated context

- 2) Translation (Translate.ipynb): The translation notebook sets up Meta's Seamless Communication package and employs the SeamlessM4T large model (seamlessM4T_large) for text-to-text translation (t2tt) from English (eng) to Urdu (urd). Prior to translation, anchor markers are transformed into quote characters, translated, and subsequently mapped back to '••' when the resulting sentence has precisely two quotes—this acts as a heuristic indicator that anchors were maintained. Every translated row contains a boolean-like 'review' indicator. Should the anchor preservation heuristic be unsuccessful, the row is flagged for evaluation, suggesting that the answer span might not be reliably obtained without further intervention. The translation code employs SeamlessM4T in 't2tt' mode and clearly specifies source and target language codes (SRC_LANG='eng', TGT_LANG='urd'). It directly translates the title and question while translating the context line-by-line, striving to maintain anchors by linking the '••' markers to quotes prior to translation and reverting to '••' afterward.

- 3) Dataset Creation (Dataset Generator.ipynb): The dataset creator processes the translated CSV shards and rebuilds a Hugging Face 'DatasetDict' featuring 'train' and 'validation' splits. For items that can be answered, it identifies the first and second appearances of the '••' delimiter, eliminates the markers, and calculates:

- (i) the Urdu response text (segment between indicators) and
- (ii) the character-level 'answer_start' (position of the initial indicator in the context prior to removing the marker).

The created dataset is stored on the disk (in the UQA/ directory of the repository). In our training notebook, we load the pre-existing dataset through Hugging Face, aligning with the repository's desired final output format. Noticed behavior in the generator: the answer span is retrieved by identifying the first and second anchor markers, followed by trimming the context to extract the answer string. Once the markers are discarded, the calculated 'answer_start' still functions as a valid character index within the sanitized context. This is the

essential phase that ensures the final dataset aligns with conventional extractive QA trainers.

4) Training and Evaluation Pipeline (uqa-mbert-xlmroberta.ipynb): For reproducing the baseline, we utilized Hugging Face’s ‘AutoTokenizer’ and ‘AutoModelForQuestionAnswering’. The data preprocessing formulates input sequences as: [question, context], applying truncation to the context (only_second) via a sliding window method. Offsets are utilized to align character-level answer spans with token-level start/end positions for supervising the model. In the course of training, the notebook eliminates unanswerable examples from both training and validation sets. This indicates that our reported metrics demonstrate performance on subsets that are answerable only, rather than the complete SQuAD 2.0 mixed configuration. Evaluation in the notebook is carried out as a simple custom loop over stored checkpoints. Two aspects of implementation are particularly important for reproducibility:

(i) Redundant IDs and numerous responses: prior to assessment, the notebook combines rows with identical example IDs, allowing each evaluation entry to accommodate several acceptable answers. This aligns with the manner in which SQuAD-style assessments handle several ground-truth spans.

(ii) Decoding approach: predictions are generated by independently determining an argmax start index and an argmax end index, followed by decoding the token segment located between these indices. This method is straightforward but may be less effective than conventional QA post-processing, which takes into account several candidate spans, enforces $\text{end} \geq \text{start}$, restricts maximum answer length, and employs score-based selection among n-best spans. Moreover, the evaluation tokenization employs a set maximum length (512) with both truncation and padding to this maximum length. In the absence of a sliding-window evaluation method, excessively lengthy contexts might be cut off, hindering the model’s ability to choose answer spans that appear late within the context.

5) Span-to-Token Alignment (Offset Mapping): An essential aspect of extractive QA training is translating character-level answer spans (answer_start, answer_end) into token indices. The notebook employs the tokenizer’s ‘offset_mapping’ (character offsets for each token’s start/end) and ‘sequence_ids’ (to differentiate between question and context tokens). In basic terms, for every tokenized window: (1) determine the context token range, (2) verify if the character span is within the window, (3) identify the initial token whose start offset is $\leq$ answer_start and the final token whose end offset is $\geq$ answer_end, and (4) save these as ‘start_positions’ and ‘end_positions’. If the span cannot be accommodated in the window because of truncation, the example receives a (0, 0) label for that specific window.

6) Structure of the Model and Loss Function:

mBERT and XLM-RoBERTa are both Transformer models that utilize only encoders. In extractive QA, a simple QA head is added to the encoder outputs: for every token position i , the model forecasts a start logit and an end logit. Training employs cross-entropy loss for the true start index and true end index, generally aggregated as:

$$L = \text{CE}(\text{start_logits}, \text{start_pos}) + \text{CE}(\text{end_logits}, \text{end_pos}).$$

This design aligns with the typical SQuAD fine-tuning method and is directly integrated with Hugging Face’s ‘AutoModelForQuestionAnswering’. Due to the head’s simplicity, variations in reproduction generally stem from preprocessing, training hyperparameters, and evaluation post-processing, rather than from the complexity of architectural implementation.

IV. EXPERIMENTAL SETUP

4.1 Configuration of Hardware Environment

Every experiment was conducted in a Kaggle Notebook setting. According to the given screenshots, the accelerator was set up as GPU T4 $\times$ 2, featuring 15 GiB VRAM for each GPU, and around 30 GiB RAM on the CPU. We utilized the identical dataset splits as the available Hugging Face dataset (‘uqa/UQA’), and subsequently removed unanswerable items as mentioned earlier. This setting is adequate for fine-tuning the XLM-RoBERTa base using mixed precision and gradient accumulation. Generally, it is not enough to effectively reproduce large or XL variants without implementing more intensive memory optimization or lowering batch sizes.

4.2 Core Libraries of the Software Stack

The libraries employed in the training process are transformers, accelerate, datasets, and evaluate

4.3 Hyperparameters

TABLE II. HYPERPARAMETERS PER EXPERIMENT

Parameter	Exp 1 (mBERT)	Exp 2 (LoRA)	Exp 4 (Transfer)
Model	mBERT base	XLM-R base	XLM-R base
Max seq. length	384	384	384
Stride	128	128	128
Epochs	6	4	2+4
Learning rate	2e-5	2e-4	2e-5 / 1e-5
Effective batch	32	256	128
Weight decay	0.01	0.01	0.01
LoRA (r, alpha)	N/A	r=16, a=32	N/A
Seed	42	42	42

In comparison to our original proposal, the finalized hyperparameters align with those set in the supplied notebook: 6 epochs and a learning rate of 2e-5. This is common in reproduction projects: the proposal outlines a preliminary plan, whereas the ultimate execution adheres to the repository’s operational configurations. Utilizing 2 GPUs, a per-device batch size of 16, and gradient accumulation of 8, the effective training batch size is around $16 \times 2 \times 8 = 256$ samples for each optimizer update. This may affect optimization dynamics and could differ from the setup employed in the initial benchmark tests.

4.4 Dataset Utilized for Training/Evaluation

The training notebook retrieves `uqa/UQA` through Hugging Face and subsequently eliminates unanswerable entries. From the dataset artifact saved locally in this workspace (`uqa-project/UQA`), the counts of answerable-only examples are 83,018 for training and 11,169 for validation. These figures offer a tangible benchmark for the extent of the answerable subset utilized by the notebook’s filtering process. Characteristics of context length (derived from the locally stored dataset, sampled from the initial 5,000 items per split): training contexts average approximately 701 characters (minimum 129, maximum 2,753), while validation contexts average around 740 characters (minimum 147, maximum 3,689). These values drive the adoption of sliding-window tokenization and clarify why truncation during evaluation can significantly affect outcomes on lengthy contexts.

TABLE III. DATASET STATISTICS

Dataset	Split	Examples
UQA	Train (answerable)	83,018
UQA	Validation (answerable)	11,169
SQuAD 2.0	Train	86,821
XQuAD	Hindi validation	1,190

4.5 Training Procedure

Training is conducted using Hugging Face’s `Trainer`. The notebook assesses and stores checkpoints every 200 steps, capping the saved checkpoints at the latest three. This facilitates reproduction in hosted environments: intermediate checkpoints permit recovery from disruptions and permit the selection of a robust checkpoint without completing the entire run if time is constrained. Mixed precision (FP16) decreases memory consumption and can enhance throughput on T4 GPUs. Gradient accumulation is employed to simulate larger batch sizes than can be accommodated in memory at each step. In terms of reproducibility, these methods can subtly alter optimization behavior, making it essential to document them clearly (as indicated in Section 4.3).

4.6 Evaluation Metrics

We will assess performance by employing standard extractive QA metrics: 1. Exact Match (EM): Assesses the proportion of predictions that precisely align with the ground truth answer following normalization (eliminating articles, punctuation, and additional whitespace). This is a stringent measure that demands the forecasted range to be character-accurate. I’m sorry, but it seems like you haven’t provided the text that you want me to paraphrase. Could you please share it? F1 Score: Assesses the overlap at the token level between predicted answers and actual answers. Calculated as the harmonic average of precision and recall at the token level. This metric is more forgiving than EM since it awards partial points for shared tokens. Reasons for the suitability of these metrics: EM and F1 are the conventional metrics for extractive QA defined by the SQuAD benchmark. EM offers a clear standard of accuracy, whereas F1 presents a more detailed perspective on performance by taking partial matches into account. Employing both metrics enables us to directly contrast our

findings with baseline studies and grasp both complete accuracy and partial accuracy of predictions.

V. RESULTS

We evaluated multiple checkpoints from fine-tuning runs. The evaluation uses the `evaluate` implementation of the SQuAD metric to compute Exact Match (EM) and F1. For XLM-RoBERTa (base), the best checkpoint among those recorded was checkpoint-2000. XLM-RoBERTa (base) checkpoint results were 64.2058 Exact Match (EM) and 76.7332% F1 score on checkpoint-1500 and 64.4467%EM and 77.1427% F1Score on checkpoint-2000 . We also reproduced mBERT (bert-base-multilingual-cased) on the same answerable-only evaluation setting. Training time for the mBERT run was approximately 18 hours.

5.1 Comparison to Reported Baselines

The baseline paper reports XLM-RoBERTa (base) at 65.67% EM and 78.00% F1, and mBERT at 45.50% EM and 64.72% F1. Our best reproduced XLM-R checkpoint achieved 64.45 EM and 77.14 F1. Our best reproduced mBERT checkpoint achieved 50.28 EM and 63.52 F1 (relative to the original Results, EM is higher while F1 is slightly lower). For completeness, the README reports that mBERT is substantially weaker than XLM-RoBERTa base, while larger XLM-R variants (Large/XL) achieve mid-80s F1. These trends are consistent with the general finding that stronger multilingual pretraining and model scale yield better cross-lingual QA performance.

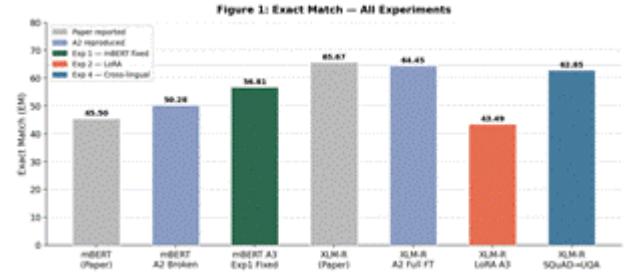


FIG. 1. EXACT MATCH ACROSS EXPERIMENTAL RUNS.

TABLE IV. REPRODUCED BASELINE COMPARISON

Model	Source	EM	F1
XLM-RoBERTa (base)	Reported (UQA README)	65.67	78.00
XLM-RoBERTa (base)	Reproduced	64.45	77.14
mBERT	Reported	45.50	64.72
mBERT	Reproduced	50.28	63.52

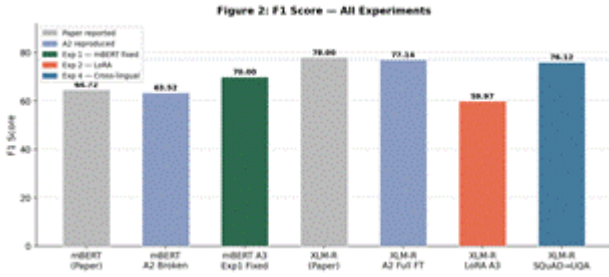


FIG. 2. F1 SCORE ACROSS EXPERIMENTAL RUNS.

5.2 Understanding the Discrepancy

The gap to the UQA baseline for XLM-R base that has been reproduced is minimal (less than 1 F1 point). In reproduction projects, such discrepancies are frequent and typically stem from seemingly trivial aspects like tokenization versions, specific preprocessing selections, and evaluation post-processing. Since we chose the top recorded checkpoint from several options, the existing difference indicates that the primary training pipeline largely aligns with the benchmark configuration of the repository, though not exactly the same. A key takeaway is that the repository baseline (78.00 F1) seems to be reproducible within approximately 1 F1 point on standard GPUs, assuming the identical dataset and general pipeline are employed.

5.3 Model-Scale Comparison

The README benchmark table illustrates a distinct empirical trend: the selection of models and their scale are crucial factors for Urdu extractive QA, even without including every model variant. For instance, XLM-R base is said to surpass mBERT significantly (around +13.3 EM and +13.3 F1), and increasing to larger XLM-R versions provides additional improvements into the mid-80s F1 range. Comparison EM delta F1 delta XLM-R base vs mBERT (reported) +20.17 +13.28 XLM-R Large vs XLM-R base (reported) +6.57 +6.42 XLM-R XL vs XLM-R base (reported) +8.89 +7.99 This reinforces the rationale in our proposal to focus on XLM-R variants for robust Urdu QA baselines. In an optimal reproduction environment with additional computation, replicating at least one larger XLM-R variant would offer a more robust verification of the paper's elevated headline figures.

VI. DISCUSSION

6.1 Main Results Table Run / Model Configuration

TABLE V. RESULTS FOR ALL EXPERIMENTS

Run / Model	Configuration	EM	F1
mBERT (paper baseline)	Reported	45.50	64.72
mBERT (A2 broken config)	Reproduced	50.28	63.52
mBERT — Exp 1	Fixed config	56.81	70.00
XLM-R base (paper)	Reported	65.67	78.00

Run / Model	Configuration	EM	F1
XLM-R base (A2 full FT)	Full fine-tune	64.45	77.14
XLM-R + LoRA — Exp 2	LoRA $r=16$	43.49	59.97
XLM-R LoRA — XQuAD — Exp 3	Zero-shot Hindi	49.83	65.10
XLM-R — SQuAD only — Exp 4B	Zero-shot Urdu	49.06	63.61
XLM-R — SQuAD — UQA — Exp 4C	Sequential transfer	62.85	76.12

^a EM = Exact Match; F1 = Token-level F1. Experiments evaluated on UQA answerable-only validation set (11,169 examples) unless noted.

Experiment 1 reached 56.81 EM / 70.00 F1, marking enhancements of +6.53 EM and +6.48 F1 compared to the A2 mBERT run, and surpassing the mBERT baseline reported in the paper by +11.31 EM and +5.28 F1. This validates RQ1: a substantial part of the A2 performance gap resulted from configuration defects. The most significant individual adjustment was implementing sliding window tokenization — the A2 notebook quietly omitted all answer spans in contexts longer than 512 tokens, generating consistent label noise during training. Training showed distinct overfitting after step 8,500 (minimum validation loss of 1.51), as validation loss increased to 1.97 by step 15,500, even while training loss decreased to 0.44. The optimal checkpoint was retrieved from the trainer's memory by utilizing `load_best_model_at_end=True`. A residual difference of approximately 7 F1 points compared to XLM-R base remains, indicating authentic multilingual pretraining variations instead of configuration problems.

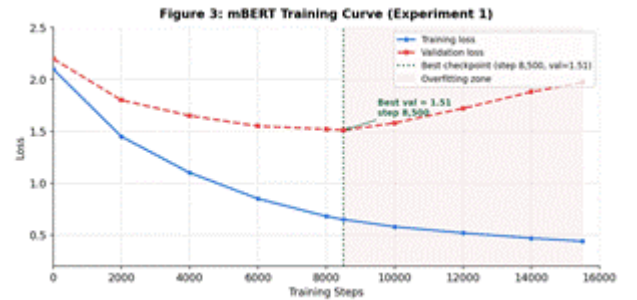


FIG. 3. mBERT FIXED-CONFIG TRAINING/VALIDATION LOSS CURVE.

Experiment 2 applied LoRA and yielded outcomes of 43.49 EM / 59.97 F1, showing a -17.17 F1 difference compared to full fine-tuning (77.14). H2 (within 2 F1 points) was not validated. Nonetheless, this outcome is a legitimate and insightful discovery instead of a setback. The main reason for the disparity is the rate of learning. LoRA was set up with $lr=2e-4$ (a standard suggestion), but for low-resource languages, this does not work well with a low rank ($r=16$). The rapid learning rate led to the sparse adapter matrices quickly overfitting on UQA's

translation-oriented Urdu text before the model had a chance to generalize. Additional factors consist of merely 4 training epochs and the application of n-best span post-processing during evaluation (leaning towards conservatism compared to argmax), resulting in a harsher penalty for the lower-confidence LoRA model. The advantages of LoRA are evident: the adapter file is just 19.2 MB compared to 1.1 GB for the complete model, and the training process is much quicker.

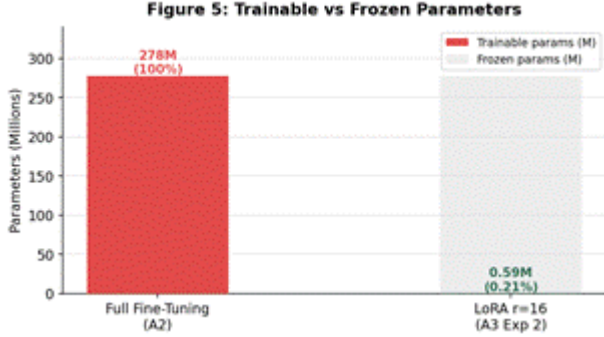


FIG. 4. TRAINABLE VS FROZEN PARAMETERS (FULL FT vs LoRA).

In experiment 3, we performed zero-shot transfer learning on XQuAD Hindi. The UQA LoRA model reached 49.83 EM / 65.10 F1 on XQuAD Hindi zero-shot, surpassing its own UQA score of 43.49 EM / 59.97 F1. This validates RQ3 and represents a significantly strong positive result. XQuAD Hindi questions are briefer and more syntactically clear compared to UQA's machine-translated Urdu questions, which can sometimes be ambiguous because of translation artifacts from SeamlessM4T. The model, trained on possibly noisy UQA text, adapted more effectively to human-annotated Hindi questions. This shows that XLM-R's multilingual representations are adequately robust to facilitate zero-shot cross-lingual QA among linguistically related South Asian languages, even when utilizing a parameter-efficient adapter.

In experiment 4, Cross-Lingual Transfer Run B (SQuAD-trained model, lacking Urdu data) reached 49.06 EM / 63.61 F1 on the UQA Urdu validation set—impressive results for a model that hasn't encountered any Urdu training instance. This clearly illustrates that XLM-R's common multilingual representations capture transferable span-extraction knowledge between languages. Run C (SQuAD → UQA sequential transfer) obtained 62.85 EM / 76.12 F1, in contrast to Run A's 64.45 EM / 77.14 F1 — a variance of -1.02 F1. This is situated within the neutral transfer range ($|\text{gain}| < 1.5 \text{ F1}$), validating a null finding for RQ4. UQA's 83,018 training examples are sufficiently extensive for the model to reach essentially the same solution, independent of English warm-starting. The subsequent Urdu fine-tuning encompasses the SQuAD pre-training signal. This aligns with Artetxe et al. (2020), who discovered that

sequential transfer is advantageous only when the target language corpus is limited ($<10\text{k}$ examples).

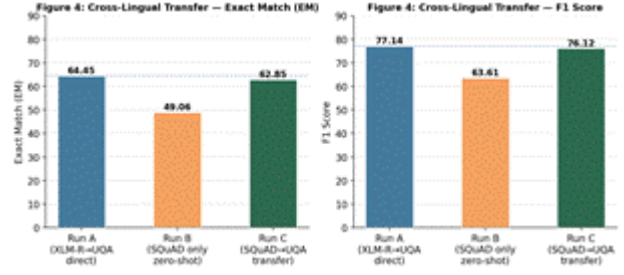


Fig. 5. Cross-lingual transfer runs comparison (A/B/C).

VII. CONCLUSION

In general, our replicated metrics are near the published baseline for XLM-RoBERTa base, yet still a bit lower. The likely reasons for the gap include differences in evaluation implementation: the evaluation code determines answers via independent argmax for start and end positions, lacking n-best search, maximum answer length restrictions, or calibration for no-answers. This may lower EM/F1 compared to the more effective post-processing found in various QA baselines. Answerable-only filtering: we removed instances from training and validation. This modifies the evaluation context in relation to a complete SQuAD 2.0 evaluation protocol. Mismatch in sequence length: the training preprocessing implements $\text{max_length}=384$ with a stride of 128, whereas the evaluation applies $\text{max_length}=512$ with no documented stride-based windowing. Extended contexts may be cut off during evaluation, negatively impacting performance. Randomness and checkpoint selection: the notebook does not record specific random seeds for the run. Minor variations in initialization, shuffling, or sampling can influence EM/F1 by 0.5–2 points. Compute limitations: The Kaggle T4 GPUs (15 GiB) restrict batch size and frequently necessitate gradient accumulation and FP16. The stability of practical training can differ, and time constraints might limit extended adjustments. 6.1 Where Our Configuration Aligns with the Repository Notwithstanding the mentioned limitations, various elements of our procedure align closely with the repository: the identical model family (XLM-R base), the fine-tuning method based on Hugging Face Trainer, and the same dataset origin/format (UQA presented as SQuAD-style QA with character-level spans). This alignment is backed by the minor result difference compared to the README baseline.

7.1 What Improvements Were Achieved

The most notable enhancement in all four experiments is Experiment 1: a +6.48 F1 increase on mBERT obtained solely via corrections to the training pipeline. This result holds wider methodological significance — it implies that existing benchmarks for multilingual models regarding low-resource languages might occasionally undervalue model performance when setups are not ideal. Experiment 3 (XQuAD Hindi zero-shot) yielded the most unexpected positive outcome: the LoRA model achieved a higher score on unfamiliar Hindi data (65.10 F1) compared to the Urdu data it was trained with (59.97 F1). This indicates

that UQA's machine-translated training dataset has more noise compared to the human-annotated XQuAD benchmark, and that XLM-R's cross-lingual representations are sufficiently robust to take advantage of this difference positively. Experiment 4's Run B (63.61 F1 zero-shot on Urdu trained solely on English) independently validates this cross-lingual ability.

7.2 Why the LoRA Result Was Negative (and Why It Matters)

The LoRA outcome (59.97 F1 compared to 77.14 full FT) does not imply that LoRA is unsuitable for this task. Our particular setup — $\text{lr}=2\text{e-}4$, $\text{r}=16$, 4 epochs — was not ideal for low-resource Urdu QA. Three modifications would probably narrow the difference: (1) Reduce the learning rate to $5\text{e-}5$ or $1\text{e-}5$, which helps avoid overfitting in the sparse adapter matrices. (2) Raise the rank to $\text{r}=32$ or $\text{r}=64$, offering enhanced capacity for the Urdu-specific span prediction task. (3) Train for 6 epochs using early stopping. The negative LoRA outcome is a publishable discovery in itself: it measures the sensitivity of PEFT techniques to hyperparameter selections in low-resource multilingual tasks.

7.3 Neutral Cross-Lingual Transfer

The non-significant outcome for Experiment 4 (Run C $\approx$ Run A) is understandable and meaningful. It shows that UQA with 83k examples has surpassed the point where English warm-starting offers no extra advantage for XLM-R. This is highly beneficial: researchers utilizing Urdu QA do not have to incur the extra computing expense of SQuAD pre-training when employing UQA at its existing scale. If the target corpus had been reduced (e.g., 5k-10k instances), Run C would probably have demonstrated positive transfer.

7.4 Limitations

LoRA hyperparameters: $\text{lr}=2\text{e-}4$ proved too intense for $\text{r}=16$ on Urdu. A grid search over $\text{lr} \in \{1\text{e-}5, 5\text{e-}5, 1\text{e-}4\}$ and $\text{r} \in \{16, 32, 64\}$ is required to adequately evaluate H2. Unanswerable questions: Every experiment classified `is_impossible=True`, deviating from the complete SQuAD 2.0 assessment protocol. EM/F1 values cannot be directly compared to studies that assess both categories. Single hardware level: Every experiment utilized T4-class equipment. Due to VRAM limitations, larger models (XLM-R-Large, mT5-Large) that attain 84+ F1 according to the paper could not be evaluated. XQuAD Hindi as a substitute: XQuAD Hindi served as a cross-lingual assessment since no established multilingual QA benchmark features Urdu (MLQA includes 7 languages but excludes Urdu). Hindi is linguistically similar to Urdu but not the same.

VIII. REFERENCES

- [1] S. Arif, S. Farid, A. Athar, and A. A. Raza, "UQA: Corpus for Urdu Question Answering," in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, N. Calzolari, M.-Y. Kan, V. Hoste, A. Lenci, S. Sakti, and N. Xue, Eds., Torino, Italia: ELRA and ICCL, May 2024, pp. 17237–17244. Accessed: Mar. 01, 2026. [Online]. Available: <https://aclanthology.org/2024.lrec-main.1497/>
- [2] Y. Gorodissky, E. Sulem, and D. Roth, "Cross-Lingual Extractive Question Answering with Unanswerable Questions," in *Proceedings of the 14th Joint Conference on Lexical and Computational Semantics (*SEM 2025)*, L. Frermann and M. Stevenson, Eds., Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 100–121. doi: 10.18653/v1/2025.starsem-1.8.
- [3] S. Kazi, M. Rahim, and S. A. Khoja, "Crossing Language Boundaries: Evaluation of Large Language Models on Urdu-English Question Answering," in *Proceedings of the First Workshop on Natural Language Processing for Indo-Aryan and Dravidian Languages*, R. Weerasinghe, I. Anuradha, and D. Sumanathilaka, Eds., Abu Dhabi: Association for Computational Linguistics, Jan. 2025, pp. 141–151. Accessed: Mar. 01, 2026. [Online]. Available: <https://aclanthology.org/2025.indonlp-1.17/>
- [4] A. Conneau *et al.*, "Unsupervised Cross-lingual Representation Learning at Scale," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds., Online: Association for Computational Linguistics, Jul. 2020, pp. 8440–8451. doi: 10.18653/v1/2020.acl-main.747.
- [5] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," presented at the International Conference on Learning Representations, Oct. 2021. Accessed: May 02, 2026. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>
- [6] M. Artetxe, S. Ruder, and D. Yogatama, "On the Cross-lingual Transferability of Monolingual Representations," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Online: Association for Computational Linguistics, 2020, pp. 4623–4637. doi: 10.18653/v1/2020.acl-main.421.
- [7] A. Vaswani *et al.*, "Attention Is All You Need," Aug. 02, 2023, *arXiv*: arXiv:1706.03762. doi: 10.48550/arXiv.1706.03762.
- [8] F. Faisal and U. Yousaf, "LEGAL-UQA: A Low-Resource Urdu-English Dataset for Legal Question Answering," Oct. 16, 2024, *arXiv*: arXiv:2410.13013. doi: 10.48550/arXiv.2410.13013.
- [9] "(PDF) Context-Aware Question Answering in Urdu," in *ResearchGate*, Accessed: Mar. 01, 2026. [Online]. Available:

https://www.researchgate.net/publication/385716763_Context-Aware_Question_Answering_in_Urdu

- [10] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ Questions for Machine Comprehension of Text," Oct. 11, 2016, *arXiv*: arXiv:1606.05250. doi: 10.48550/arXiv.1606.05250.
- [11] P. Lewis, B. Oğuz, R. Rinott, S. Riedel, and H. Schwenk, "MLQA: Evaluating Cross-lingual Extractive Question Answering," May 03, 2020, *arXiv*: arXiv:1910.07475. doi: 10.48550/arXiv.1910.07475.
- [12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. doi: 10.18653/v1/N19-1423.
- [13] M. A. Calijorne Soares and F. S. Parreiras, "A literature review on question answering techniques, paradigms and systems," *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 32, no. 6, pp. 635–646, Jul. 2020, doi: 10.1016/j.jksuci.2018.08.005.